

Core Use Cases and Gap Analysis

<https://datatracker.ietf.org/doc/html/draft-chen-oauth-agent-authz-use-cases-01>

<https://github.com/Maisy-ML/Agent-Authorization-Use-Cases>

Meiling Chen(China Mobile)

PART 01

Personal & Consumer Scenarios

Use Case 1: Personal Digital Assistant

01 Scenario

Users issue high-level natural language commands to AI agents, which autonomously decompose goals into sequential sub-tasks and API calls.

Example: "Plan a Saturday picnic"

1. Check calendar availability
2. Fetch weather forecast
3. Search maps for nearby parks
4. Book a picnic shelter
5. Add event to your calendar

```
[User] --"Plan Picnic"--> [Agent]
      |--> (1) Access [Weather API]
      |--> (2) Pause & Ask User: "Park A or B?" (Step-up Auth)
      |--> (3) Access [Booking API]
```

02 Authorization Requirements

1. Intent-to-Permission Mapping

Translate abstract goals into specific, minimal, just-in-time permissions dynamically.

2. Interactive Consent Loops

Agent must pause mid-execution to request approval for sensitive sub-actions.

3. Task-Level Revocation

Enable one-click revocation of all permissions tied to a single task context.

03 Current Gaps

1. Paradigm Mismatch

OAuth' s "pre-authorization" model cannot grant dynamic, intent-aware permissions.

2. No Interactive Flow Standards

No protocol exists for agents to "pause" and request real-time user decisions.

3. Coarse-Grained Revocation

Current standards only support revoking by app, not by task context.

Use Case 2: Smart Home & Automation

01 Scenario

A central home hub agent acts as the "brain" of the household, autonomously orchestrating IoT devices based on predefined rules and user preferences.

Example: "Good Morning" Routine (7 AM)

- 1. Gradually brighten bedroom lights to 80%
- 2. Set thermostat to 22°C (72°F) for comfort
- 3. Trigger coffee maker to start brewing automatically

```
[User] --"Morning Routine"--> [Agent]
      |--Condition: 7:00 AM--> [Smart Blinds]
      |--Condition: User Awake--> [Coffee Maker]
```

02 Authorization Requirements

1. Persistent Delegation

Seamless daily operation after one-time setup, no repeated user authentication prompts.

2. Granular & Conditional

Allow adjusting light brightness but block door unlocking; restrict access to specific times (e.g., 7 AM only).

3. Bulk Revocation

Critical for scenarios like home resale—revoke all hub permissions with a single action.

03 Current Gaps

1. Scope Explosion

OAuth consent screens become unusable when presenting thousands of device-specific scopes to users.

2. Missing Policy Context

Standard OAuth grants define "what" access is allowed, but not "when" or "under what conditions" (e.g., time of day).

3. No Bulk Revoke Standard

Lack of a universal API to revoke all tokens for a specific user or device instance.

Use Case 3: Agent as User's Representative to 3rd-party Tools

01 Scenario & Impact

AI agents act as direct proxies for users, interacting with services designed for humans. This mismatch can trigger anti-abuse systems or violate platform policies, often penalizing the user for the agent's behavior.

⚠ Real-World Example

A research agent autonomously downloads 400+ academic papers in 15 minutes. The publisher's anti-bot system flags this as suspicious activity, resulting in the **permanent suspension of the user's personal account**—even though the user was unaware of the agent's actions.

02 Critical Requirements

1. Agent-User Distinction

Services must reliably detect if a request comes from a human or an automated agent.

2. Agent-Specific Policies

Apply stricter rate limits or quotas to agents without impacting human user experience.

3. Constrained Delegation

Grant limited, scoped permissions (e.g., "max 50 actions/hour") instead of full user access.

03 Current Gaps & Risks

1. No Standard Agent Identity

OAuth 2.0 lacks a standard claim to explicitly signal "automated agent" status, leading to misclassification.

2. Binary Scope Limitation

Standard scopes are "all-or-nothing" and cannot encode dynamic constraints like rate limits or data caps.

3. Confused Deputy Problem

Service providers struggle to distinguish legitimate agent activity from malicious abuse, risking account bans.

Conclusion: To enable safe agent-to-service interactions, we need identity protocols that explicitly represent agent status and support fine-grained, dynamic authorization constraints.

Use Case 4: Agent Accessing OS Resources as User's Proxy

01 Scenario

Scenario Context

AI agents on user endpoints require direct access to local resources—such as file systems, registry settings, or hardware—to execute device-level tasks seamlessly.

Practical Example

User request: “Clean up my downloads folder.” The agent needs read/write access to `/home/user/downloads`` to scan, evaluate, and delete unneeded files, demanding precise, scoped filesystem permissions.

02 Authorization Requirements

1. Granular Resource Scope

Limit access to only the exact files or system APIs required for the task, avoiding over-privileged agent instances to minimize attack surfaces.

2. Task-Bound Permissions

Permissions are ephemeral: granted for the task’s duration only and automatically revoked upon completion or failure to prevent lingering access risks.

3. User-Mediated Escalation

A secure, real-time workflow for user approval when the agent requests privileges beyond pre-configured scopes, ensuring human-in-the-loop control.

03 Gap Analysis

1. Architectural Misalignment

OAuth 2.0 and OIDC are web-centric protocols designed for inter-service API access, not for managing local OS process permissions on endpoints.

2. Missing Intent-to-Permission Bridge

No standardized protocol exists to translate an agent’s high-level task intent into secure, fine-grained, and temporary OS permission grants.

3. Risk of Over-Authorization

Developers often resort to broad, persistent permissions (e.g., admin rights) to bypass gaps, creating critical vulnerabilities if the agent is compromised.

Use Case 5: First Connection to a Service with No Pre-Authorization Channel

01 Scenario

Description

A user attempts to connect their agent to a service that lacks a formal authorization server or any browser-based authentication endpoint—common in simple headless services or legacy systems.

Example

An agent needs to write to a private note-taking REST API. It requires a credential with a specific scope (`notes.write`), a defined expiration lifetime, and the ability for the user to revoke access instantly—all without a browser redirect.

02 Authorization Requirements

1. Pre-Channel-less Bootstrapping

Enable secure first-time pairing even when neither the service nor the agent can initiate a browser-based interaction or redirect flow.

2. User-Initiated & Constrained

The user must explicitly trigger access, define granular scopes (e.g., read-only), and set short-lived expiration timelines.

3. Revocable by Design

Credentials must be instantly invalidatable by the user to prevent orphaned access and mitigate breaches.

03 Gap Analysis

1. RFC8628's Core Limitation

Device authorization flow still assumes a human-accessible verification page exists, which is not true for many headless or isolated services.

2. Missing Standard for "Cold Start"

There is no universal, secure protocol for granting scoped, revocable access in a "zero-trust, no-pre-channel" initialization scenario.

3. Insecure Anti-Patterns

Developers fall back to long-lived API keys in environment variables—these are over-broad, hard to rotate, and create severe security risks.

PART 02

Enterprise & Business Process Scenarios

Use Case 6: Complex Business Process Automation

01 Scenario

A task flows through a sequence of specialized AI agents, where each agent executes a discrete, critical step in a larger operational workflow.

Example: Insurance Claims Processing

Intake Agent → Verification Agent → Adjudication Agent → Payout Agent — each step depends on the secure handoff of the prior agent's output.

02 Authorization Requirements

1. Verifiable Delegation Chain

Cryptographically validate the entire authorization path to ensure no tampering or unauthorized access across agent handoffs.

2. Per-Step Least Privilege

Strictly limit each agent's permissions to only the data and actions required for its specific step in the chain.

3. Persistent Audit Context

Anchor the entire process to a unique, immutable claim_id for end-to-end traceability and compliance.

03 Gap Analysis

1. No Native Multi-Hop Support

OAuth 2.0 is designed for direct User-to-Agent delegation, not chained Agent-to-Agent handoffs, causing token validity issues in multi-step workflows.

2. Missing Context Standardization

No standardized token field for securely propagating critical workflow context (e.g., claim_id), leading to insecure, non-interoperable custom implementations.

Key Takeaway: Traditional OAuth lacks the primitives to secure distributed, multi-agent business process automation. Securing these workflows requires new standards for verifiable multi-hop delegation and standardized context propagation.

Use Case 7: Coordinated Task Group

Scenario	Authorization Requirements	Gap Analysis
01. Core Mechanism A coordinating agent decomposes a high-level request into granular subtasks, then dynamically assembles a dedicated team of specialized sub-agents to execute them in parallel or sequence. 02. Health Assistant Example To assess wellness: split into data collection (vitals), condition prediction (ML model), and recommendation (diet/exercise) — assigning each discrete step to a specialized sub-agent.	1. One-Time Group Authorization Coordinator secures a single grant for the entire team, eliminating redundant, per-member auth flows that slow execution. 2. Member-Specific Least Privilege Each sub-agent is strictly limited to its unique scope (e.g., read-only data access), preventing unauthorized privilege escalation. 3. Dynamic Late Binding Sub-agents added mid-task automatically inherit valid permissions under the parent group’s authorization, no re-authentication needed.	1. Scalability Bottleneck N sub-agents require N separate authorization server calls, creating significant latency and overwhelming infrastructure at scale. No standard for proxying group auth. 2. Token Model Limitation OAuth 2.0 tokens are designed for individual identities. There is no standardized way to encode group membership or collective resource scopes. 3. Risk: Orphaned Permissions No atomic revocation means terminating each member individually, risking lingering access.

Use Case 8 & 9: DNSSEC Maintenance & Managed Services

01 Scenario

DNSSEC Maintenance

Automates DS record updates during cryptographic key rollover. It relies on a strict delegation chain: Registrar → Registrar Agent → Registry Agent to maintain trust.

Managed Services

A master service integrator consolidates infrastructure from multiple sub-providers. This model necessitates efficient cross-domain trust establishment and bulk resource authorization.

02 Authorization Requirements

1. Verifiable Multi-hop Delegation

Create an immutable audit trail for trust passing through multiple administrative hops.

2. Granular, Least-Privilege Access

Restrict permissions to specific domain names, record types, and administrative actions.

3. Cross-Domain Trust Propagation

Seamlessly extend authorization across distinct organizational and security domains.

4. Bulk Resource Authorization

Request and grant access to a collection of resources in a single workflow.

03 Gap Analysis

1. No Standard for Multi-hop

OAuth 2.0 only natively supports direct two-party delegation, failing to model complex multi-agent trust chains.

2. Lack of Bulk Revocation

No standardized way to revoke access for a group of tokens or tasks, forcing tedious single-entry operations.

3. Absence of Batch Requests

No protocol-level support for requesting access to multiple distinct resources or scopes in one request.

PART 03

Security & Administrative Scenarios

Use Case 10: Automated Security Incident Response

01 Scenario

When a threat is detected (e.g., a compromised laptop), an automated security agent must act instantly to contain the breach before damage escalates.

Required Automated Actions:

1. Revoke all active sessions for the compromised user.
2. Isolate the infected device from the corporate network.
3. Temporarily suspend the user's account to block new access.

02 Authorization Needs

1. Pre-granted, system-level privileges to act decisively without manual approval.
2. A **Global Token Revocation API** to invalidate `/tokens` for a user in real time.

03 Critical Gap

Current OAuth standards (e.g., RFC7009) only support *single-token* revocation endpoints, which are completely inadequate for active security incidents.

Why It's a Failure Point:

Issuing thousands of individual API calls to revoke each token is too slow and unreliable. Attackers exploit this delay to maintain access and exfiltrate data.

Summary of Common Requirements

Key Requirements for Agent-Centric Authorization in OAuth 2.0

Requirement 1: Dynamic Scope & Intent Expression

Need for scopes that can dynamically reflect user intent, support fine-grained conditions (e.g., time-bound, event-driven), and include non-functional constraints (e.g., rate limits). This prevents scope explosion and enables precise, context-aware access control for agents.

Requirement 2: Flexible & Diverse Interaction Methods

Require standardized authorization flows that are not solely dependent on browser-based UIs. Need to support headless environments, local OS processes, and provide standardized channels for mid-task step-up authentication.

Requirement 3: Native Agent & Group Identity Support

Need for native token-level identifiers for "AI Agents (Agent ID)" and "Agent Groups". The framework must natively distinguish between human users and autonomous agents, and support permission management at a collaborative group level.

Requirement 4: Secure Multi-hop & Cross-Domain Delegation

Require native support for multi-hop delegation chains (User -> Agent A -> Agent B -> Service C) and cross-domain delegation. A standardized mechanism is needed to securely pass audit and security context across each hop.

Requirement 5: Dynamic Lifecycle & Advanced Revocation

Need for bulk and atomic revocation capabilities. Token lifecycles should be dynamically bindable to external business states (e.g., SLAs, task completion, security incident triggers) for agile and context-aware revocation.

Analysis of Existing OAuth Extensions for Agent Authorization

01 RAR (RFC 9396)

SOLVED: Enables fine-grained, parameterized access requests (e.g., "transfer \$50 to X"), moving beyond generic scopes.

GAP: Static by design; unable to natively adapt to the dynamic, non-deterministic actions of autonomous agents.

OPTIMIZATION: Integrate "Bounded Capabilities" to dynamically translate agent intent into RAR JSON with time/budget constraints.

02 DPoP (RFC 9449)

SOLVED: Prevents token replay attacks by cryptographically binding tokens to the client's private key.

GAP: Assumes fixed key pairs, making it incompatible with agent migration or dynamic sub-agent creation.

OPTIMIZATION: Enable dynamic key rotation via WIMSE and support "Multi-hop DPoP" for secure token derivation to sub-agents.

03 Transaction Tokens

SOLVED: Securely propagates user context and attributes across internal microservices for consistent authorization.

GAP: Designed for trusted internal networks, not suitable for untrusted third-party agent interactions.

OPTIMIZATION: Introduce "Attenuating Agent Tokens" for one-way permission reduction and embed PSEA for execution proofs.

04: CIMD draft-ietf-oauth-client-id-metadata-document

Solved

Dynamic Trust Foundation

By defining `client\_id` as a resolvable URI, CIMD establishes a decentralized foundation for dynamic trust, eliminating static identifier constraints.

Plug-and-Play Onboarding

Agents no longer need pre-registration. Authorization Servers dynamically fetch metadata via URI, enabling seamless "plug-and-play" integration.

Simplified Key Lifecycle

Agents expose public keys via `jwks\_uri`, allowing AS to retrieve updated keys on-demand and significantly reducing manual key rotation overhead.

Gap

Lack of Agent Context

Standard metadata fields (e.g., redirect URIs) are app-centric, lacking attributes to describe agent autonomy, risk levels, or operational boundaries.

Public URI Dependency

Ephemeral or internal sub-agents often lack a stable public URI to host CIMD documents, breaking the discovery mechanism for transient agents.

Missing Delegation Traceability

CIMD verifies individual identity but cannot express "who created whom," making it impossible to trace trust chains in multi-hop agent scenarios.

Optimization

Agent-Specific Metadata

Introduce `agent\_type`, `max\_autonomous\_budget`, and `human\_in\_the\_loop` to enable granular risk assessment by Authorization Servers.

Inline/Vouched Metadata

Allow ephemeral agents to carry metadata directly in requests or have it "vouched for" by a trusted parent agent, removing the public URI requirement.

Trust Chain Assertions

Add `parent\_agent\_id` or integrate OpenID Federation-style nested statements to verify the legitimacy of delegation paths across agent networks.

Open Discussion

A Three-Layered Authorization Model for Agents

Ensuring Trust and Safety in Autonomous Agent Operations

01 Grant Layer: User Consent

Solved:Initial resource access permissions.

-Proves that "at 9 AM, permission was granted".

Tech:OAuth 2.0 (**Session-scoped**- proves scope granted).

02 Condition Layer: Runtime Integrity

Solved:Longevity & revocation checks.

-Proves that "between 9 AM and 3 PM, the Agent's status remained healthy and permission was not revoked".

Tech:WIMSE (**Environment-scoped**- verifies uncompromised state).

03 Execution Layer: Action Validation

Solved:Verifying genuine user intent.

-Proves that "at 3 PM, a human/system approved this specific action".

Tech: **Action-scoped**- binds payload cryptographically.

Six Key Questions for Agent Authorization

Q1: Identity & Trust

How do we establish and verify an AI agent's "machine identity" and runtime environment security, independent of users? Is this beyond OAuth's scope, better suited for groups like WIMSE?

Q2: Fine-grained Bounding & Intent

How to translate vague user intent into precise, API-level authorization contracts enforceable by a gateway? Is RAR sufficient for dynamically binding AI actions to user goals?

Q3: Multi-hop Delegation & Attenuation

How can parent agents generate strictly attenuated sub-tokens for child agents without central AS calls? What are the bottlenecks of Token Exchange for high-frequency machine delegation chains?

Q4: Decoupled Delivery & Data Sovereignty

How to separate the "authorization control channel" from the "data delivery channel" to prevent data retention? Is GNAP's multi-party interaction model better suited than OAuth for this separation?

Q5: Execution Assurance & Auditability

How to require cryptographically verifiable proofs of execution (PSEA) for high-risk operations? Should SCITT mechanisms be integrated to ensure compliance and non-repudiation?

Q6: Economic Settlement & Consent

How to bind authorization tokens with micropayment channels and legally binding consent for agent transactions? Which standards body should govern this intersection of identity and value?